

CMS Workload Management Selection

Abstract

CMS reviewed its workflow management system used during Run 1, 2, and 3 to determine if it is suitable for the high-luminosity era of the LHC (HL-LHC). The review found that support of the current system is insufficient and the system thus not viable for HL-LHC. The committee recommended to investigate ATLAS PanDA, LHCb DIRAC systems in addition to developing a custom CMS system as options. This document describes the selection among the three options.

Introduction

The production workflow management (WM) system, WMCore/WMAgent, of CMS was developed for Run 1 of the LHC. It handles Tier-0 activities, i.e. calibration, archiving to tape, data distribution to Tier-1 sites, prompt reconstruction, and all offline production requests of data processing and Monte Carlo generation/simulation.

User analysis needs are handled by a second WM system, the CMS Remote Analysis Builder (CRAB). Both systems are layered on top of an HTCondor pool (GlobalPool) which acquires compute resources via glide-in WMS. Components beside the GlobalPool are shared between the two systems but they are separate systems, maintained and operated by different teams. Previous Offline and Computing (O&C) coordinators tried to combine the two WMs a couple of years ago but due to limited interest and manpower in the support groups this stalled. CRAB support and development will end during long shutdown 3 (LS3) of the LHC scheduled for 2026 to 2030.

During 2024 CMS reviewed the production workflow management system, to check its viability for HL-LHC, how to best add user analysis support, and the required effort for this. During the review it became clear that the custom WMCore/WMAgent system had become difficult to maintain and a significant boost of effort would be required for it to be viable for HL-LHC. The review committee concluded in 2025 and recommended [\[1\]](#) to investigate the workflow management system used by ATLAS, PanDA, and the one used by LHCb, DIRAC, in addition to building a new custom system for CMS. The development should be handled as a project with resource-loaded schedule, including maintenance and operations effort. Especially long-term support until the mid 2040s should be considered in the decision.

This document describes the initial investigations into the three WM options; the Workflow Management Workshop in autumn of 2025 to collect and clarify WM requirements from the various stakeholders; the second round of investigations; and the O&C Week in spring of 2026 to review the investigation results, discuss, and conclude on the plan forward.

The investigation options

The ATLAS experiment built a custom workflow management system for Run 1. The system, PanDA, is being revised and extended to keep up with the changing Grid infrastructure and to meet the processing needs of the experiment. ATLAS uses PanDA for experiment-wide production, data processing and Monte Carlo generation/simulation, and grid-based user analysis. Tier-0 activities are handled by a legacy setup.

LHCb developed DIRAC to handle production and analysis workflows on the grid. The system has been rewritten to keep up with changing middleware. Since a couple of years it has an interface to the Rucio data management system. Other experiments, like Belle II, CTAO, and JUNO, have adopted DIRAC for

their workflow management needs by customizing generic modules.

Rewriting the CMS workflow management system is a significant effort even if many parts can be extracted from the current system. The lack of institutional interest and long-term commitment may lead us back into a similar situation during Run 4 or 5.

Initial investigations

The WM review committee presented a preliminary version of their findings to the CMS O&C management team during a retreat [\[2\]](#) in June of 2025. During discussions in the following weeks it became clear that to choose between the three proposed options a more detailed understanding of each option would be needed.

For each option a "champion" within O&C was identified and asked to investigate with the goal to identify problematic issues, reduced functionality compared to the current system, new features that could benefit CMS, how to interface with existing components, where work would be needed, and to estimate the required effort. The deadline for this was set to end of September 2025.

The first round of investigations established that all three options were technically plausible, but with very different implications for CMS.

For the CMS-led revamp option, a first architectural proposal was developed around retaining HTCondor and GlideinWMS as the execution substrate while replacing the current monolithic and long-lived agent model with a modular control plane and many short-lived micro-agents. The motivation was that the present architecture is not sustainable for the order of (10^4) concurrent work-queue elements expected at HL-LHC scale. The proposal introduced a central workflow database, request translators for different entry points, a workflow orchestrator, a pool of access points, and per-work-unit micro-agents responsible for local execution control, retries, and bookkeeping. The main expected advantages were improved scalability, better fault isolation, reduced operational burden through automation, and easier support for heterogeneous resources such as HPC and cloud facilities. A proof-of-concept workflow orchestrator and job splitter were also developed to validate these ideas.

The PanDA investigation concluded that PanDA is a technically viable option for Monte Carlo production and user analysis, and likely also for Tier-0 processing, although the latter was not demonstrated in production-like conditions. No hard technical blockers were identified. PanDA was found to provide native support for chain workflows, multiple input datasets, scout jobs for automatic resource estimation, and heterogeneous resources such as GPUs. The JEDI component appeared able to support CMS requirements such as lumi-based processing. At the same time, the investigation highlighted that adoption by CMS would require substantial integration work. In particular, CMS-specific plugins would have to be developed for request management, monitoring, DBS integration, and adaptation of the pilot payload and stage-in/stage-out model. The large code base, deployment complexity, and reliance on a highly optimized database backend were also noted as potential long-term sustainability concerns.

The initial DIRAC/DiracX investigations found the system attractive because of its modularity and because several experiments have already adopted it successfully. DIRAC/DiracX appeared compatible in principle with CMS needs, but important gaps were identified. In particular, work unit granularity is currently file-based, heterogeneous resource support was not yet native, and the ongoing migration from DIRAC to DiracX introduced uncertainty in the maturity timeline of the needed components. DIRAC's integration with Rucio was found to be possible, though with architectural differences compared to CMS. In Belle II, for example, jobs are generally sent where data already exists and

output registration is performed from the worker node, whereas CMS currently relies on more centralised post-job checks and registrations. No fundamental incompatibility with Rucio was identified.

The Tier-0 specific investigation showed that this use case deserves separate attention. CMS Tier-0 has important special requirements: Express and Repack workflows are created from streamer files before data is in DBS or Rucio, they run in real time on a first-come, first-served basis, and workers read inputs from CERN disk through xrootd streaming. PromptReco is created later, once data is in Rucio and DBS. ATLAS Tier-0 was found to be independent of PanDA and grid execution for historical rather than technical reasons. LHCb uses DIRAC for Tier-0-like processing at Tier-1 sites. The investigation therefore found no decisive architectural argument excluding either PanDA or DIRAC for CMS Tier-0, but it also made clear that this part of the CMS workload is operationally special and would require dedicated adaptation work in any solution.

Workflow Management Workshop in Autumn 2025

The Data Management and Workflow Management Development (DMWM) team organized a workshop [\[3\]](#) for mid September to bring all WM stakeholders to the table. The idea being that each group would tell us what they don't like or features the current WM system lacks. Each champion would have a chance to discuss and clarify the requirements most relevant, challenging, or constraining to their approach.

Second round of investigations

During the autumn O&C Week/workshop in October each champion reported their findings [\[4\]](#). No showstopper(s) were found for either option and some weak aspects were highlighted. A week later, after some discussions, CMS launched six additional investigations:

1. Investigate replacing native queuing of PanDA with our Global Pool
2. Investigate how a PanDA system adopted as is would fit CMS and make a list of all implications
3. Use a DiracX instance and explore running jobs with increased complexity
4. Identify all the DiracX components needed for core functionality/CSA, investigate readiness timeline, extra CMS effort to speed up availability, if possible, etc.
5. Investigate replacing the database in DiracX with PostgreSQL or Oracle DB
6. Work out a complete revamp option proposal

All members of the DMWM team took on an investigation or teamed up with other collaboration members on the investigation. For the new, more detailed investigations more time than for the first round was given and a deadline of end of February 2026 set.

The second-round investigations produced a much more detailed picture of the three options.

1. Replacing native provisioning of PanDA and DIRAC with the CMS Global Pool

A detailed study of replacing the native DIRAC queuing and pilot matching mechanisms with GlideinWMS and the CMS Global Pool concluded that such an integration is feasible. The report identified the main duplication of functionality between DIRAC and the Global Pool in resource provisioning and pilot/job matching. The preferred solution was a producer-queue-consumer model with two adaptor layers. In this model, a minimally modified DIRAC WorkflowTaskAgent would generate job objects and pass them to a new queue service, implemented as a lightweight DiracX service. Independent CMSCondorAgent instances would then claim jobs from that queue, translate

them to HTCondor JDL, and submit them to schedds in the Global Pool. A separate status updater service would synchronize job state back to the central queue database. This strategy was preferred because it minimized invasive changes to the DIRAC core while preserving token-based authentication and allowing CMS to retain GlideinWMS and HTCondor as the central resource management layer.

2. PanDA adopted as is: implications for CMS

The PanDA adoption study concluded that the system is technically viable for CMS production and analysis use cases, with no hard blockers identified. PanDA provides mature support for task chaining, multi-dataset workflows, automatic scouting of resource requirements, and heterogeneous resources. However, using it in CMS would have major implications. CMS would need new integration layers or plugins for workflow request handling, DBS lookup, monitoring, and payload adaptation. Integration with GlideinWMS would likely require changes in JEDI, and possibly Harvester and Pilot3, because PanDA normally performs its own brokerage and provisioning decisions. The study also recommended that CMS should not adopt iDDS as a separate component, since its functionality is being absorbed into the PanDA server. The deployment exercise further showed that PanDA is a powerful but complex ecosystem whose installation and tuning require significant expertise.

3. DiracX instance with increasingly complex jobs

A hands-on evaluation of a DiracX instance was carried out to explore execution of increasingly complex jobs. This investigation helped confirm that the framework is promising and modern in structure, but also that CMS-specific execution patterns would require further work. In particular, more realistic chains of tasks, data-handling patterns, and integration with CMS operational conventions need to be exercised before one can draw strong conclusions about production readiness for CMS-scale workflows.

4. DiracX components needed for core functionality

A dedicated readiness study reviewed which DiracX components are required for CMS core functionality, and when they are expected to become available. Configuration and deployment tooling were already in place, with deployment supported via Helm charts. Workflow support was partially available. However, several critical components were still under development or not yet scheduled firmly: the Resource Status System, the Transformation System, the Data Management System integration needed by CMS, the Request Management System, and possibly a generic Production System. The study concluded that a complete and CMS-usable DiracX stack was unlikely to be available in time unless CMS contributed development effort. A contribution at the level of roughly 1–2 FTE was estimated to be capable of significantly accelerating the availability of the key missing components and of influencing their design in directions important to CMS.

5. DiracX database backend and technical comparison

The database-focused investigation compared PanDA, DIRAC/DiracX, and WMCore from both architectural and code-quality perspectives. PanDA was found to have the largest code base, DIRAC a smaller one, and DiracX substantially smaller still. PanDA and DIRAC/DiracX both support modern deployment patterns including Kubernetes. From the software metrics inspected, WMCore and DIRAC appeared better documented internally than PanDA, while PanDA showed somewhat higher average cyclomatic complexity. The investigation of DiracX database layers highlighted more specific concerns. DiracX uses both a relational SQL layer and a non-relational OpenSearch-like layer. The SQL abstraction was found to have limitations for large result sets, with materialization in memory rather than streaming, and portability issues were identified in practice despite SQLAlchemy-based abstraction. In particular, PostgreSQL support had proven difficult, and Oracle support existed in a

separate, non-integrated layer. The study also pointed out that current DiracX/LHCb schemas do not match CMS granularity requirements such as explicit run/lumi-aware work units. This investigation therefore raised important questions about backend portability and whether CMS-specific schema extensions would be required.

6. Complete revamp option proposal

The revamp investigation matured the earlier architectural ideas into a more complete proposal for a CMS-led next-generation WMS. The central design choice was to preserve HTCondor and GlideinWMS while replacing the current WMAgent architecture with a workflow control plane built from clear service boundaries and a large number of short-lived micro-agents. A single central workflow database would store workflow documents and work-queue elements. Request translators would map Tier-0, production, and analysis requests into a common internal format. A stateless workflow orchestrator would prioritize work, select access points, and coordinate with data-placement services. Each work-queue element would then be managed by a dedicated micro-agent responsible for HTCondor job expansion, local state, retry logic, and output handling. The proposal emphasized resilience, horizontal scaling, data-aware scheduling, and support for all major CMS workflow classes in a single architecture. It also outlined a staged implementation plan based on proof-of-concept, prototype deployment alongside the existing system, and progressive consolidation toward Run 4 readiness.

Latest update: Offline and Computing Week in Spring 2026

The Spring O&C Week/workshop [\[5\]](#) was dedicated to reviewing results of the additional investigations, discussing ways forward, and reaching a consensus on the next steps. Investigators reported what they did and what they found during three sessions. In the afternoon of the day following the main sessions a round table discussion took place.

The additional investigations had clarified several key points. PanDA appeared technically strong and feature-rich, especially for analysis workflows and heterogeneous resources, but adoption would require substantial CMS-specific integration effort and operational expertise. DIRAC/DiracX appeared modular and promising, and partnership with its developers could reduce the burden of building and maintaining a CMS-only system, but important components were not yet fully mature on the timescale needed by CMS and some core assumptions, such as work granularity and backend support, would need adaptation. The revamp option offered the best alignment with CMS-specific requirements and with preserving the GlideinWMS Global Pool model, but it would require sustained project organisation and dedicated development resources over multiple years.

Conclusions

The CB Subcommittee on Phase-2 Effort had discussed O&C needs with each country/institute, had asked them to consider joining an O&C activity, and report on the effort at which they would support the WM project. The representatives spoke first about their interest and potential effort. One institute expressed plans to start a new CMS computing activity and was preparing a proposal for the funding agency. One country announced a reduction in WM development support. All other countries did not want to participate.

Then everybody else in the room and on Zoom voiced their view and opinion. A consensus then slowly emerged:

1. to not pursue a PanDA-based option further
2. to follow the review committee recommendation and preserve the GlobalPool and glide-in

WMS

3. to explore with the DIRAC/DiracX team a partnership on a workflow solution including the GlobalPool/glide-in WMS
4. to start realizing a micro-agent based revamp in case item 3 cannot be realized

The O&C coordinators were charged with following up on item 3.

Appendices

<!--

Context: Following the review committee's recommendation, CMS convened to evaluate workload management (WM) solutions for the High-Luminosity LHC (HL-LHC) era. The process was composed of a series of investigations coordinated with periodical meetings and converged into a series of reports and discussions at the Spring O&C Week 2026.

-->

Previous meetings and in-depth material

- Review committee report on the viability of WMCore/WMAgent for HL-LHC
- Initial technical investigations of PanDA, DIRAC/DiracX, and a CMS-led revamp
- Follow-up studies on Tier-0, Rucio integration, Global Pool integration, and DiracX readiness

Overview tables

Option	Status	Key Characteristics
DIRAC/DiracX	Modular implementation	~10 integrated modules; promising architecture; important components still maturing
PanDA	Mature and proven at scale	Plugin-based; supports heterogeneous resources; complex integration scenarios
CMS-led evolution	Rework proposal	Built around GlideinWMS + HTCondor; micro-agent concept

- **DIRAC/DiracX** offers modularity and partnership potential but requires integration work for CMS-specific needs and additional development to meet CMS timescales.
- **PanDA** is mature, flexible, and feature-rich but complex to integrate; it is particularly strong for analysis workflows and heterogeneous resources.
- **CMS-led** evolution maximizes control and alignment with HL-LHC goals but demands significant new development; micro-agent proof-of-concepts demonstrate feasibility and possible efficiency gains.

Strategic direction: strong emphasis on automation, fault tolerance, and operational sustainability. Regardless of the chosen path, key CMS capabilities such as monitoring, DBS integration, token authentication, and compatibility with the Global Pool must be preserved or exceeded.

DIRAC/DiracX Architecture

Core component quick notes:

- **Services:** Request listeners and handlers
- **Agents:** Periodic cron-like tasks
- **Executors:** Trigger-based asynchronous handlers
- **Transformation System:** Job splitting, work/data binding, assignment
- **Production System:** Multi-step workflow chain management
- **Database:** MySQL-based today; ongoing evolution in DiracX
- **Data Management:** DIRAC File Catalog with Rucio support under development and validation

Key Strengths:

- Modular service-oriented design
- Clear migration path toward more modern interfaces in DiracX
- Existing adoption by several experiments beyond LHCb
- Feasible integration with the CMS Global Pool through adaptor services

Key Limitations:

- No native heterogeneous resource support (GPU/ARM) yet
- File-based job granularity remains restrictive for some CMS use cases
- DiracX migration timeline depends on available developer effort
- Database portability and schema granularity remain concerns for CMS-scale usage

PanDA (ATLAS) Architecture

Core components quick notes:

- **PanDA Server:** Central hub for requests and job lifecycle
- **JEDI:** Job splitting, early binding, and data/resource decisions
- **Harvester:** Resource provisioning and pilot dispatch
- **PanDA Pilot v3:** Runtime execution with modular staging
- **iDDS:** Workflow dependency/DAG management, increasingly integrated into core PanDA functionality

Key Strengths:

- Plugin-based modularity across core components
- Flexible job splitting by event, file, size, or lumi block
- Native support for heterogeneous resources (GPU, ARM)
- Mature token and X.509 authentication support
- Intelligent rebrokering and retry across sites
- Strong support for analysis-style workflows

Key Limitations:

- Significant integration effort required for CMS-specific services and interfaces
- Large and operationally complex software stack
- Deployment and tuning require substantial expert involvement
- Current production operation relies heavily on optimized database and infrastructure choices

"CMS-Led" Architectural Proposal

Design Principles

- Serve all use cases: data processing, simulation, private workflows
- Build around existing GlideinWMS + HTCondor partnership

- Macro-components with clear boundaries and plugin-based interactions
- Users specify ETA and physics priority; the system maps these to fair-share and scheduling policies
- Dynamic feedback loop between global resource pool and orchestrator
- Fault isolation via short-lived micro-agents

Macro-Component Architecture

Component	Responsibility	Development Status
Request Translator	Translate user requests into workflow documents; normalize Tier-0, production, and analysis inputs	Evolution of existing request interfaces
Workflow Orchestrator	Prioritize work-queue elements, select access points, coordinate execution and data staging	Proof-of-concept / proposed
Data Placement Manager / Broker	Aggregate placement requests; interface with Rucio and metadata services	Evolution of current data services
Micro-Agents	Short-lived execution controllers per work unit: payload prep, submission, retries, local bookkeeping	New development
Workflow Database	Central state for workflow documents and work-queue elements	New development

Execution Flow

1. A workflow document is created and stored in the central workflow database.
2. The orchestrator reads the document and produces work units or work-queue elements.
3. A micro-agent is launched for each work unit through an HTCondor access point.
4. The micro-agent expands the work unit into jobs, tracks execution locally, handles retries and output processing.
5. Feedback from execution and data access is sent back to the orchestrator to refine scheduling and placement decisions.

Fault Tolerance Strategy

- Failures are isolated at the micro-agent level
- Retry logic is automatic and can include site-aware blacklisting
- Micro-agents store local recovery state in embedded databases
- Persistent failures are escalated to operators; transient failures are handled automatically
- Data access failures can trigger fallback strategies and revised placement decisions

Proposed Development Timeline

1. **Phase 1 (now–2026):** Proof-of-concept for micro-agents and service interactions
2. **Phase 2 (2026–2027):** Production-oriented prototype tested with real workloads alongside the current system
3. **Phase 3 (2027–2028):** Scaling, integration of production interfaces, and consolidation toward CSA28
4. **Pre-Run 4:** Integration of analysis support and operator tooling

Table for technical comparison

Feature	DIRAC/DiracX	PanDA	CMS Proposal
Modularity	Microservice-based / evolving modular services	Plugin-based per component	Macro-components + microservices

Job Granularity	File-based	Event/file/size/lumi block	Work units
Workflow Dependencies	Production requests and transformation chains	DAG-based / chained tasks	Schema-defined workflow documents
Heterogeneous Resources	Not native yet	Supported	Designed via HTCondor primitives
Authentication	Migrating to tokens in DiracX	Tokens + X.509	Token-compliant
Data Bookkeeping	Integrated plus Rucio interfaces	External (Rucio/FTS)	Rucio + WM data abstraction layer
Database Backend	MySQL, evolving support in DiracX	Oracle, PostgreSQL, MySQL	Central workflow DB + per-agent SQLite
Late Binding	Requires integration work	Supported/adaptable	Core design principle

Discussions summary

Interest statements

- **FCC:** Showed strong interest in DIRAC at the December IRIS-HEP workshop
- **DUNE/Rubin:** Monitoring the CMS decision closely; current systems are not seen as sustainable long-term; willing to collaborate if CMS follows a compatible path

Q/A on "CMS-led" architecture requirements

Question	Response
Work unit sizing: Orchestrator does not know CPU requirements upfront	Scouting work units run first to characterize resources; results inform subsequent unit creation
Micro-agent scalability: One agent per work unit could overwhelm the system	Micro-agents are ephemeral and lightweight; HTCondor handles large-scale queuing and matchmaking
Error diagnosis: Hard to distinguish config/site/code failures	Telemetry, log analysis, and AI/LLM tools could support automated diagnosis; standardized error reporting is needed
Priority/ETA handling: Users may request infeasible deadlines	ETA is treated iteratively; the system and operators adjust scope and priority dynamically
Task/step chains: How are multi-step workflows handled?	Chains can be represented either inside one work unit or across multiple work units
Data locality changes: What if data moves after dispatch?	The data-placement layer provides updated locality information; micro-agents feed execution experience back to the orchestrator

Monitoring & Ops requirements

- Debugging and failure analysis remain the main challenge regardless of the chosen WM system
- Standardized error reporting is needed
- AI/LLM integration was proposed for log analysis and operator decision support
- Operator tools and feedback loops must be designed from the start

Tier-0 Processing

- **DIRAC:** Used by LHCb for Tier-0-like activities
- **PanDA:** Not currently used by ATLAS for Tier-0, largely for historical reasons
- **CMS:** Has special Tier-0 requirements, including streamer-file based workflow creation and CERN-centric data access patterns

User Analysis Support

CMS features to preserve regardless of choice:

1. LumiSection completeness accounting
2. DBS integration for output registration
3. ASO direct placement to user disk quota
4. Submission simplicity
5. Aggregate monitoring and dashboards
6. Token-based authentication roadmap

DM considerations

- Integration with Rucio appears possible in both PanDA and DIRAC/DiracX
- CMS differs from both systems in where data registration and movement logic are centralized
- The interplay between data management and workload management remains a key integration topic

<!--

Effort and contributions

The CMS Workflow Management (WM) working group convened during the Spring O&C week to assess the community's capacity and roadmap for a sustainable, scalable WM system for HL-LHC.

Representatives from CERN, Europe, the U.S., Asia and the U.K. confirmed a mixed availability of effort: the U.S. CMS operations program can contribute up to ~2 FTE, Germany and Spain could add 2-3 FTE contingent on grant success, while other sites (Italy, Taiwan, China, India, Pakistan) have limited or no dedicated WM resources but are willing to support operations and testing.

The discussion converged on three key messages:

- WM must be treated as a formal project with clear milestones, deliverables, and a dedicated manager
- a comprehensive, solution-agnostic requirements document—building on the September workshop—should be produced before any technical decision
- while the CMS-owned evolution of the current system remains an option, there is strong interest, especially from the U.S. and Germany, in exploring a partnership with the DiracX effort, provided it can accommodate the required scale and user-facing requirements

Conclusions and action items

Plans

- **Plan A:** Pursue a partnership with the DIRAC/DiracX team to develop a CMS-compatible solution that preserves the Global Pool and GlideinWMS model
- **Plan B:** Continue with the CMS-led revamp based on a workflow control plane and micro-agents if a satisfactory partnership cannot be realized on the required timescale

Action items

- Finalize the requirements document
- Engage with the DiracX team to keep a CMS-specific path open
- Identify a project umbrella and leadership structure able to coordinate the effort across institutions and funding agencies
- Continue technical prototyping of the revamp architecture to reduce risk in case Plan B becomes necessary

-->

References

1. CMS Workload Management Review Report 2025, <https://cms-docdb.cern.ch/cgi-bin/DocDB/ShowDocument?docid=14913> [↵](#)
2. CMS Offline and Computing Management Meeting "F2F", <https://indico.cern.ch/event/1521355/timetable/#all.detailed> [↵](#)
3. 2025 DMWM Workshop, <https://indico.cern.ch/event/1554074/timetable/> [↵](#)
4. Fall 2025 Offline Software and Computing Week, <https://indico.cern.ch/event/1550918/> [↵](#)
5. CMS Spring O&C Week, <https://indico.cern.ch/event/1612910/> [↵](#)